

Cluster: EII



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS**

**A Project Report On
Project/Thesis Management System (TPMS)**

Submitted By:

Prabesh BC (PUL080BCT054)
Shreeyut Thapa (PUL080BCT080)
Subesh Yadav (PUL080BCT084)

Submitted To:

Department of Electronics & Computer Engineering

In partial fulfillment of the requirements for the degree of
Bachelor of Engineering in Computer Engineering

DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING

Lalitpur, Nepal

August, 2026

Declaration of Authorship

This is to certify that the minor project entitled “**Project/Thesis Management System (TPMS)**” submitted for the degree of Bachelor of Engineering in Computer Engineering comprises our original work and no part of this work has been used for the award of another course or degree. We declare that this work is our own. If any part of the work is not our own, we have indicated it by acknowledging the source of that part or those parts of the work with proper citations.

Name of the Student

Signature

Prabesh BC

Shreeyut Thapa

Subesh Yadav

Date: _____

Page of Approval

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

The undersigned certifies that they have read and recommended to the Institute of Engineering for acceptance of a project report entitled “**Project/Thesis Management System (TPMS)**” submitted by **Prabesh BC (PUL080BCT054)**, **Shreeyut Thapa (PUL080BCT080)**, and **Subesh Yadav (PUL080BCT084)** in partial fulfillment of the requirements for the Bachelor’s degree in Computer Engineering.

.....

Supervisor

Full Name of Supervisor

Designation

Department of Electronics and Computer
Engineering,
Pulchowk Campus, IOE, TU.

.....

Internal Examiner

Full Name

Assistant Professor

Department of Electronics and Computer
Engineering,
Pulchowk Campus, IOE, TU.

.....

Head of Department, Prof. Dr. Ram Krishna Maharjan
Department of Electronics and Computer Engineering,
Pulchowk Campus, IOE, TU.

Date of approval: _____

Copyright

The authors have agreed that the library, Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering may make this project report freely available for inspection. Moreover, the authors have agreed that permission for extensive copying of this project report for scholarly purposes may be granted by the supervisors who supervised the work recorded herein or, in their absence, by the Head of the Department wherein the project report was done. It is understood that recognition will be given to the authors of this report and the Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering for any use of the material of this project report. Copying, publication or the other use of this report for financial gain without the approval of the Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering, and the authors' written permission is prohibited.

Request for permission to copy or to make any other use of the material in this report in whole or in part should be addressed to:

Head

Department of Electronics and Computer Engineering

Pulchowk Campus, Institute of Engineering, TU

Lalitpur, Nepal.

Acknowledgements

The authors would like to express their sincere gratitude to the project supervisor, **Department of Electronics and Computer Engineering, Pulchowk Campus**, for continuous guidance, constructive feedback, and invaluable support throughout the entire development lifecycle of this project.

The authors are deeply grateful to the **Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering, Tribhuvan University** for providing the academic framework, laboratory infrastructure, and institutional context that made this project possible.

Sincere thanks are extended to all faculty members and staff of DOECE who participated in User Acceptance Testing sessions and provided feedback that significantly improved the system's usability. Gratitude is also expressed to all fellow classmates and student peers who voluntarily validated the student-facing workflows.

Finally, the authors wish to express their deepest appreciation to their families for constant moral support and encouragement throughout the duration of this program.

Abstract

The Project/Thesis Management System (TPMS) is a web-based platform developed for the Department of Electronics and Computer Engineering (DOECE), Pulchowk Campus, Institute of Engineering, Tribhuvan University. The system digitalizes and automates the complete administrative and evaluative lifecycle of Bachelor Minor and Major projects and Master research theses across six academic programs, replacing existing manual, paper-based workflows.

The platform enforces a five-role hierarchy—Maintainer, Coordinator, Supervisor, Student, and External Examiner—governed by Role-Based Access Control (RBAC) with program-scoped degree isolation. A Global Engagement Guard prevents students from enrolling in multiple active project groups simultaneously. An automated Conflict-of-Interest Filter ensures that project supervisors cannot serve as defense evaluators for their own supervised projects. The system implements the DOECE evaluation schemes for Minor projects (50 marks), Major projects (100 marks), and Master Theses (300 marks) through a dynamic rubric engine.

Key technical contributions include a Puppeteer headless browser pipeline generating official A4 PDF grade sheets with automated number-to-words mark conversion, and a SheetJS-powered Excel bulk import pipeline with pre-commit anomaly detection. A program-scoped audit trail ensures institutional accountability.

The system is implemented using React 18 with Vite 5 on the frontend, Node.js with Express.js on the backend, and PostgreSQL 16 managed via Prisma 5 ORM. Evaluation through unit testing (36 tests, 100% pass rate), integration testing, security testing, and User Acceptance Testing (4.47/5.0 satisfaction score) confirmed functional completeness and operational readiness.

Keywords: *Role-Based Access Control, Project Management System, Evaluation Rubric Engine, PDF Generation, Node.js, React, PostgreSQL*

Contents

Declaration of Authorship	ii
Page of Approval	iii
Copyright	iv
Acknowledgements	v
Abstract	vi
Contents	ix
List of Figures	x
List of Tables	xi
List of Abbreviations	xii
1 Introduction	xiii
1.1 Background	xiii
1.2 Problem Statement	xiii
1.3 Objectives	xiv
1.3.1 General Objectives	xiv
1.3.2 Specific Objectives	xiv
1.4 Rationale	xv
1.5 Scope	xv
1.6 Limitations	xvi
1.7 Outline of the Report	xvi
2 Literature Review	xvii
2.1 Review of Related Theory	xvii
2.1.1 Role-Based Access Control (RBAC)	xvii
2.1.2 Representational State Transfer (REST)	xvii
2.1.3 JSON Web Tokens (JWT)	xvii
2.1.4 Object-Relational Mapping (ORM)	xvii

2.1.5	Agile Scrum Methodology	xviii
2.1.6	Headless Browser Automation and PDF Generation	xviii
2.2	Review of Related Works	xviii
2.2.1	Moodle Learning Management System	xviii
2.2.2	ProQuest ETD Management	xviii
2.2.3	Turnitin Academic Integrity Platform	xix
2.2.4	Kaur and Singh (2014)	xix
2.2.5	Sharma et al. (2019)	xix
2.3	Gap Analysis	xix
3	Methodology	xxi
3.1	Project Approach	xxi
3.2	System Architecture	xxi
3.3	System Design and Modules	xxii
3.3.1	Authentication and Authorization Module	xxii
3.3.2	Announcement and Group Formation Module	xxii
3.3.3	Coordinator Response Matrix Module	xxii
3.3.4	Evaluation Rubric Engine	xxii
3.3.5	PDF Grade Sheet Generation Module	xxiii
3.3.6	Excel Bulk Import Module	xxiii
3.3.7	Notification and Audit Module	xxiii
3.4	Technology Stack	xxiii
3.5	Development Methodology	xxiii
3.6	Data Handling	xxiv
3.7	Testing and Validation	xxiv
3.8	Deployment Setup	xxv
4	Implementation Details	xxvi
4.1	Requirements Specification	xxvii
4.1.1	Functional Requirements	xxvii
4.1.2	Non-Functional Requirements	xxviii
4.2	System Overview	xxviii
4.3	Development Environment	xxviii
4.4	System Implementation	xxix
4.4.1	Database Schema Design	xxix
4.4.2	Authentication Module	xxx
4.4.3	Group Formation and Engagement Guard	xxx

4.4.4	Conflict-of-Interest Filter	xxxix
4.4.5	Evaluation Rubric Engine	xxxix
4.4.6	PDF Grade Sheet Generation	xxxix
4.4.7	Excel Bulk Import Pipeline	xxxix
4.5	Interface and Visualization	xxxix
4.6	Deployment and Execution	xxxix
4.7	Challenges and Solutions	xxxix
5	Results and Discussion	xxxv
5.1	Results	xxxv
5.1.1	Unit Testing Results	xxxv
5.1.2	Integration Testing Results	xxxv
5.1.3	Security Testing Results	xxxvi
5.1.4	User Acceptance Testing Results	xxxvi
5.1.5	System Performance Benchmarks	xxxvii
5.2	Discussion	xxxvii
5.3	Comparative Analysis	xxxviii
6	Conclusions and Recommendations	xxxix
6.1	Conclusions	xxxix
6.2	Recommendations and Future Enhancements	xl
	References	xli
	Appendices	xliv
	Appendix A: System API Route Reference	xlvi
	Appendix B: Technology Version Reference	xlvi
	Appendix C: Database Enum Types	xlvi

List of Figures

3.1	Three-Tier System Architecture of TPMS	xxii
4.1	Entity-Relationship Diagram of TPMS Database	xxix
4.2	Level 0 Context Data Flow Diagram	xxxi
4.3	Conflict-of-Interest Filter Algorithm Flow	xxxii
4.4	DOECE Evaluation Scheme: Mark Distribution by Project Tier	xxxii

List of Tables

2.1	Comparative Analysis of Related Systems Against TPMS	xx
3.1	Technology Stack and Justification	xxiii
4.1	Functional Requirements of TPMS	xxvii
4.2	Development Environment Specifications	xxviii
4.3	Data Dictionary: User Table	xxx
4.4	Data Dictionary: ProjectGroup Table	xxx
4.5	Technical Challenges Encountered and Solutions Applied	xxxiv
5.1	Unit Test Results by Module	xxxv
5.2	User Acceptance Testing Results by Stakeholder Role	xxxvii
5.3	API Response Time Benchmarks	xxxvii

List of Abbreviations

API	Application Programming Interface
BCT	Bachelor of Computer Engineering
BEI	Bachelor of Electronics, Communication and Information Engineering
COI	Conflict of Interest
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
DOECE	Department of Electronics and Computer Engineering
ER	Entity Relationship
HTTP	Hypertext Transfer Protocol
IOE	Institute of Engineering
JWT	JSON Web Token
ORM	Object-Relational Mapping
PDF	Portable Document Format
RBAC	Role-Based Access Control
REST	Representational State Transfer
SPA	Single-Page Application
SQL	Structured Query Language
TPMS	Thesis and Project Management System
TU	Tribhuvan University
UAT	User Acceptance Testing
UI	User Interface
UML	Unified Modeling Language
UX	User Experience

1. Introduction

1.1 Background

Academic project and thesis coursework represent pivotal milestones in engineering education at Tribhuvan University (TU), Institute of Engineering (IOE), Pulchowk Campus. The Department of Electronics and Computer Engineering (DOECE) oversees compulsory Minor and Major projects for undergraduate students enrolled in Bachelor of Computer Engineering (BCT) and Bachelor of Electronics, Communication and Information Engineering (BEI) during their third and fourth academic years. Simultaneously, postgraduate scholars across four Master of Science programs—Computer Systems and Knowledge Engineering (MSCSK), Information and Communication Engineering (MSICE), Data Science and Analytics (MSDSA), and Network and Cyber Security (MSNCS)—conduct multi-semester research theses under the supervision of departmental faculty.

Historically, the administration of these academic initiatives has relied heavily on manual, paper-intensive, and fragmented processes. Departmental coordinators gather proposal drafts physically, maintain student rosters in disconnected spreadsheets, collect printed mid-term and final reports, and distribute handwritten evaluation rubrics during formal defense sessions. As student enrollment and specialized postgraduate programs have grown substantially, this legacy model has created increasing administrative strain, operational bottlenecks, scheduling conflicts, and data inconsistencies that impede the quality and accountability of academic project governance [1].

Modern higher education institutions increasingly adopt specialized academic information systems to govern collaborative workflows, enforce institutional accountability, and maintain transparent audit trails [2]. The **Project/Thesis Management System (TPMS)** was developed as an enterprise-grade, web-based platform specifically engineered to digitalize, govern, and streamline the complete lifecycle of Bachelor projects and Master theses at DOECE, Pulchowk Campus.

1.2 Problem Statement

The prevailing manual and spreadsheet-dependent methodology for project and thesis governance at DOECE presents the following systemic challenges:

P1. Decentralized Record-Keeping: Project records, student rosters, supervisor alloca-

tions, and proposal documents reside across disconnected physical binders, local spreadsheets, and unstructured email threads. This fragmentation prevents real-time visibility into project progress and milestone completion.

- P2. Absence of Engagement Controls:** No mechanism currently prevents a student from simultaneously participating in multiple project groups, leading to duplicated enrollment and unfair resource allocation.
- P3. Supervisory Conflict-of-Interest Risk:** The manual examiner assignment process lacks any systematic check to ensure that a project supervisor is not simultaneously assigned as a defense evaluator for the same project, creating a significant ethical and academic integrity risk.
- P4. Manual Mark Tabulation Errors:** Evaluation rubric scores from multiple evaluators are manually summed and transcribed, introducing arithmetic errors and inconsistencies in final grade records.
- P5. Absence of Official Document Automation:** Generation of official A4 evaluation sheets and grade documents requires hours of manual typesetting per academic cohort, delaying results and creating formatting inconsistencies.
- P6. No Audit Trail:** The current system maintains no tamper-evident log of administrative actions such as supervisor assignments, mark submissions, and group finalization decisions, preventing institutional accountability.

1.3 Objectives

1.3.1 General Objectives

The general objective of this project is to design, develop, and validate a unified, role-aware, web-based platform that automates the complete administrative and evaluative lifecycle of Bachelor projects and Master theses at the Department of Electronics and Computer Engineering, Pulchowk Campus, eliminating manual processes and enforcing institutional integrity controls.

1.3.2 Specific Objectives

- SO1.** To implement a five-role RBAC hierarchy (Maintainer, Coordinator, Supervisor, Student, External Examiner) with strict program-scoped degree isolation for Bachelor and Master programs.

- SO2.** To develop a Global Engagement Guard that prevents any student from enrolling in more than one active project group or thesis simultaneously.
- SO3.** To implement an automated Conflict-of-Interest Filter that systematically excludes a project supervisor from the pool of eligible defense examiners for their supervised projects.
- SO4.** To build a dynamic evaluation rubric engine implementing the DOECE grading schemes: Minor (50 marks), Major (100 marks), and Master Thesis (300 marks).
- SO5.** To develop a server-side Puppeteer-based PDF pipeline that generates official A4 grade sheets with automated number-to-words mark conversion.
- SO6.** To implement a SheetJS-powered Excel bulk import pipeline with pre-commit anomaly detection for large-scale student roster onboarding.
- SO7.** To create a program-scoped, immutable audit trail capturing all significant administrative and evaluative events.

1.4 Rationale

The TPMS platform directly addresses all six identified problem areas. By centralizing all project and thesis records in a relational database governed by a REST API, the system eliminates fragmented spreadsheets and physical binders. The Engagement Guard and Conflict-of-Interest Filter introduce algorithmic safeguards that replace error-prone manual checks. The automated rubric engine removes arithmetic transcription errors from mark tabulation. The Puppeteer PDF pipeline reduces grade sheet preparation time from hours to seconds. The audit trail ensures institutional accountability for every administrative decision.

Beyond operational efficiency, the system supports DOECE in maintaining compliance with IOE academic governance standards as student enrollment in specialized Master programs continues to grow [3].

1.5 Scope

The scope of TPMS encompasses the following functional boundaries:

- Management of Bachelor Minor and Major projects and Master research theses across all DOECE programs at Pulchowk Campus.
- A five-role user management system with program-scoped data isolation.

- The complete project lifecycle from group formation and proposal submission through supervisor assignment, multi-stage evaluation, and official PDF grade sheet generation.
- Bulk student roster import from structured Excel files with pre-commit anomaly detection.
- An immutable, program-scoped audit log covering all critical system events.
- In-app notification delivery for all workflow state changes.

The system does not include integration with external university ERP systems, mobile application development, or real-time video conferencing for defense sessions.

1.6 Limitations

- L1.** The current implementation is scoped to DOECE, Pulchowk Campus; multi-institution deployment would require multi-tenancy architecture changes.
- L2.** The initial deployment target is the departmental intranet; public internet exposure requires additional security hardening.
- L3.** Evaluation rubric weightages are application-layer constants; coordinator-configurable rubric editing is a planned future enhancement.
- L4.** The system does not provide integrated defense scheduling or room allocation functionality.
- L5.** Uploaded documents are stored on the application server local filesystem; cloud object storage migration is identified as a future work item.

1.7 Outline of the Report

The remainder of this report is organized as follows. Chapter 2 reviews relevant literature and theoretical background. Chapter 3 presents the project methodology, system architecture, and design. Chapter 4 details the implementation of all system modules. Chapter 5 presents testing results and discussion. Chapter 6 concludes the report and identifies recommendations for future work. References and appendices follow the main chapters.

2. Literature Review

2.1 Review of Related Theory

2.1.1 Role-Based Access Control (RBAC)

Role-Based Access Control is a security model in which system access permissions are assigned to roles rather than to individual users [4]. Users are then assigned to roles, and automatically inherit the permissions of those roles. RBAC simplifies administration and enforces the principle of least privilege, ensuring that each user can only access the functionality required for their specific institutional role. TPMS implements a five-role RBAC hierarchy—Maintainer, Coordinator, Supervisor, Student, and External Examiner—with each role mapped to a strictly scoped set of API endpoint permissions.

2.1.2 Representational State Transfer (REST)

REST is an architectural style for distributed hypermedia systems, introduced by Fielding [5], defined by six constraints: client-server separation, statelessness, cacheability, uniform interface, layered system, and optional code-on-demand. RESTful APIs communicate over HTTP using standard methods (GET, POST, PUT, PATCH, DELETE) and resource-oriented URL schemes. TPMS exposes a comprehensive RESTful API that decouples the frontend SPA from the backend service, enabling independent development and deployment of each tier.

2.1.3 JSON Web Tokens (JWT)

JSON Web Tokens are a compact, URL-safe means of representing claims to be transferred between two parties [6]. A JWT consists of three Base64URL-encoded parts: a header specifying the algorithm, a payload containing claims (e.g., user ID, role, program), and a cryptographic signature. TPMS issues HS256-signed JWTs upon successful authentication, which are transmitted as Bearer tokens in HTTP Authorization headers. The stateless nature of JWTs eliminates the need for server-side session storage, improving horizontal scalability.

2.1.4 Object-Relational Mapping (ORM)

An ORM provides a bridge between object-oriented programming models and relational database schemas, allowing developers to interact with database tables through language-native objects rather than raw SQL [7]. Prisma ORM, used in TPMS, generates a type-

safe client from a declarative schema definition, enforces foreign key relationships, manages database migrations, and provides a query builder that prevents SQL injection by parameterizing all inputs.

2.1.5 Agile Scrum Methodology

Agile Scrum is an iterative, incremental software development framework structured around short development sprints (typically 1–4 weeks), daily stand-up meetings, sprint reviews, and retrospectives [8]. TPMS development followed an Agile Scrum approach with four two-week sprints, enabling frequent stakeholder feedback from DOECE coordinators, faculty supervisors, and student representatives. This iterative model ensured that evolving departmental requirements were accommodated without large-scale architectural rework.

2.1.6 Headless Browser Automation and PDF Generation

Puppeteer is a Node.js library providing a high-level API to control Chromium over the DevTools Protocol [9]. It enables server-side rendering of HTML templates into pixel-accurate PDFs using Chromium printing capabilities. TPMS uses Puppeteer to generate official A4 evaluation sheets, replicating the visual fidelity of manually typeset documents while automating all data population and arithmetic.

2.2 Review of Related Works

2.2.1 Moodle Learning Management System

Moodle is a widely deployed open-source Learning Management System providing course management, assignment submission, and grade tracking [10]. While Moodle offers basic assignment submission and grade recording, it lacks the domain-specific features required for structured project lifecycle management: group formation state machines, multi-stage rubric-based evaluation schemes, conflict-of-interest filters, and official document generation. Its generic design necessitates extensive plugin configuration and customization for academic project governance use cases.

2.2.2 ProQuest ETD Management

ProQuest Electronic Thesis and Dissertation (ETD) management systems [11] provide institutional repositories for thesis document archiving, metadata management, and cross-institutional discovery. These systems focus on post-completion document archival and do not address the active management of the project lifecycle: group formation, supervisor allocation, evaluation rubric administration, or live mark tabulation.

2.2.3 Turnitin Academic Integrity Platform

Turnitin [12] provides originality checking, similarity scoring, and rubric-based grading tools for academic submissions. While Turnitin's GradeMark feature supports structured rubric evaluation, it is designed for individual course-level assessments rather than multi-semester project lifecycle management with role-specific dashboards, PDF grade sheet generation, and institutional conflict-of-interest enforcement.

2.2.4 Kaur and Singh (2014)

Kaur and Singh [13] proposed a web-based student project management system for Indian university engineering departments that included project registration, supervisor allocation, and progress tracking. However, their system did not address multi-stage rubric-based evaluation, automatic conflict-of-interest detection, degree-tiered marking schemes, or server-side PDF generation—all of which are critical for DOECE operational requirements.

2.2.5 Sharma et al. (2019)

Sharma et al. [14] developed a web-based academic management system for thesis submission tracking and supervisor communication. Their system provided document upload and supervisor feedback mechanisms but lacked RBAC-enforced role isolation, multi-program degree support, engagement guards for duplicate enrollment prevention, and automated official document generation.

2.3 Gap Analysis

A comparative analysis of existing related systems against TPMS is presented in Table 2.1.

Table 2.1: Comparative Analysis of Related Systems Against TPMS

Feature	Moodle	ProQuest ETD	Turnitin	Kaur & Singh	TPMS
Multi-role RBAC	Partial	No	Partial	Partial	Yes
Group formation engine	No	No	No	Partial	Yes
Engagement guard	No	No	No	No	Yes
COI filter	No	No	No	No	Yes
Multi-stage rubric eval	No	No	Partial	No	Yes
DOECE mark schemes	No	No	No	No	Yes
Server-side PDF export	No	Yes	No	No	Yes
Excel bulk import	No	No	No	No	Yes
Audit trail	Partial	No	No	No	Yes
Multi-program isolation	No	No	No	No	Yes
In-app notifications	Yes	No	No	No	Yes

The gap analysis confirms that none of the existing systems satisfies the full set of operational requirements specific to DOECE, Pulchowk Campus. In particular, the combination of RBAC-enforced multi-program degree isolation, the algorithmic Engagement Guard, the Conflict-of-Interest Filter, tiered evaluation schemes, and server-side PDF generation represents a unique technical contribution not addressed by any prior work reviewed.

3. Methodology

3.1 Project Approach

TPMS is a product-based software engineering project, developed following an **Agile Scrum** methodology with iterative two-week sprint cycles [8]. The Scrum approach was selected because DOECE departmental requirements evolved progressively throughout the project period, and frequent stakeholder feedback from coordinators, supervisors, and student representatives was essential to correctly specify domain-specific rules such as evaluation mark schemes, conflict-of-interest constraints, and engagement guards.

Development was organized into four sprints:

- **Sprint 1 (Weeks 1–2):** Requirements elicitation, system architecture design, database schema design, development environment setup.
- **Sprint 2 (Weeks 3–6):** Authentication, user management, announcement module, group formation and invitation engine.
- **Sprint 3 (Weeks 7–10):** Coordinator response matrix, supervisor assignment, evaluation rubric engine, PDF generation pipeline.
- **Sprint 4 (Weeks 11–14):** Excel bulk import, audit logging, notification system, UAT, bug fixing, and documentation.

3.2 System Architecture

TPMS adopts a **three-tier client-server architecture** enforcing strict separation between the presentation, business logic, and data persistence layers [15]. Figure 3.1 illustrates the layered architecture.

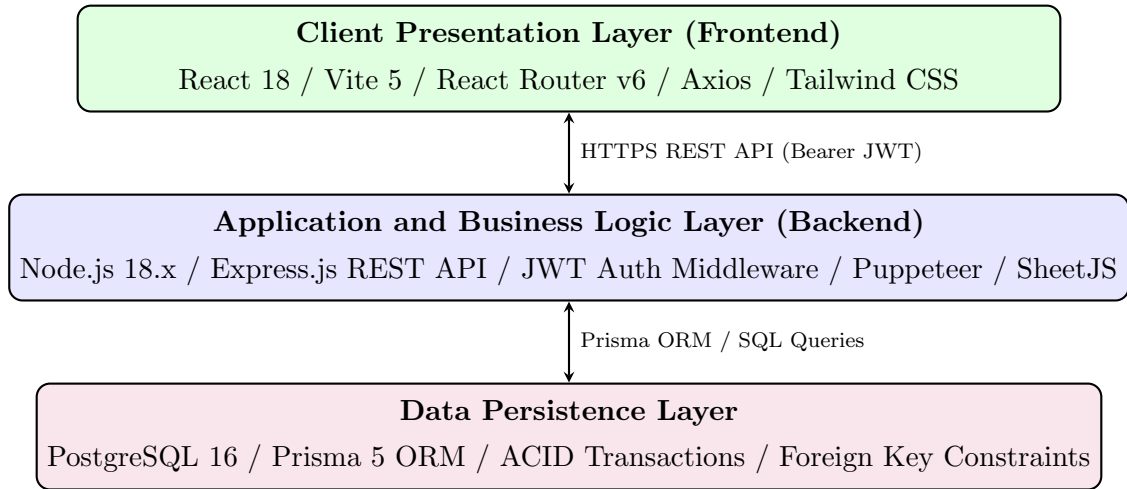


Figure 3.1: Three-Tier System Architecture of TPMS

3.3 System Design and Modules

TPMS is decomposed into the following primary functional modules:

3.3.1 Authentication and Authorization Module

Handles user login, JWT issuance and verification, password hashing with bcrypt, and role-based route protection middleware. The module enforces the five-role RBAC hierarchy and program-scoped data isolation.

3.3.2 Announcement and Group Formation Module

Enables coordinators to publish academic announcements with configurable group size limits and deadlines. Students use the announcement as the entry point to create project groups, invite peers by roll number, and manage invitation acceptance or rejection through a state machine.

3.3.3 Coordinator Response Matrix Module

Provides a deduplicated, real-time view of all submitted project groups. The module supports 1-click fuzzy supervisor matching, inline title and cluster editing, and group finalization with automatic evaluation rubric generation and email notification dispatch.

3.3.4 Evaluation Rubric Engine

Implements the DOECE mark distribution schemes for Minor (50 marks), Major (100 marks), and Master Thesis (300 marks) projects. The engine auto-generates evaluation component records upon group finalization, supports DRAFT and COMPLETED mark submission states, and enforces evaluator role matching.

3.3.5 PDF Grade Sheet Generation Module

Uses Puppeteer to render server-side HTML templates into official A4 PDF evaluation sheets, incorporating automated number-to-words mark conversion and supporting single-group and bulk multi-group export.

3.3.6 Excel Bulk Import Module

Provides a SheetJS-powered pipeline for parsing structured Excel student rosters. The module performs intra-file duplicate detection, schema validation, and cross-database conflict checking before presenting a pre-flight anomaly report to the coordinator prior to any database write.

3.3.7 Notification and Audit Module

Delivers in-app notifications for all workflow state transitions. The program-scoped audit log records all significant system events with actor ID, entity ID, event type, timestamp, and program scope.

3.4 Technology Stack

Table 3.1: Technology Stack and Justification

Layer	Technology	Justification
Frontend	React 18 / Vite 5 React Router v6 Tailwind CSS	Component-based SPA with fast HMR build tooling Client-side role-guarded routing Utility-first styling for rapid responsive UI
Backend	Node.js 18.x Express.js 4.x Puppeteer 21.x SheetJS (xlsx) 0.18.x bcryptjs / jsonwebtoken	Non-blocking I/O suitable for concurrent REST API serving Minimal, middleware-composable HTTP framework Headless Chromium for server-side PDF rendering Excel parsing for bulk roster import Secure password hashing and JWT management
Database	PostgreSQL 16 Prisma 5 ORM	Production-grade relational DBMS with ACID compliance Type-safe query builder with schema-driven migrations

3.5 Development Methodology

Agile Scrum ceremonies conducted throughout the project included:

- **Sprint Planning:** Functional requirements were decomposed into sprint-level user stories with acceptance criteria at the start of each sprint.
- **Daily Stand-up (3-member team):** Brief daily synchronization on progress, blockers, and plan adjustments.
- **Sprint Review:** Functional demonstration to the project supervisor at the end of each sprint for feedback and course correction.
- **Sprint Retrospective:** Internal team review of process quality and identification of improvement actions.

Requirements were classified into two categories:

- **Functional Requirements (FR):** System behaviors directly exposed to users (e.g., group formation, mark submission, PDF generation).
- **Non-Functional Requirements (NFR):** Quality attributes (e.g., API response time under 300 ms, bcrypt work factor of 10, program-scoped data isolation).

3.6 Data Handling

All persistent data is stored in a PostgreSQL 16 relational database managed via Prisma ORM. Input data enters the system through validated REST API endpoints (JSON body or multipart form for file uploads). All user-supplied string inputs are parameterized through Prisma query methods, preventing SQL injection. Uploaded PDF documents are stored in a server-side filesystem directory hierarchy organized by program and academic year. Password data is hashed with bcryptjs at work factor 10 before persistence; plaintext passwords are never stored.

3.7 Testing and Validation

Testing was conducted across three levels:

- **Unit Testing:** Individual controller functions and utility modules were tested using the Jest framework with Supertest for HTTP-level simulation.
- **Integration Testing:** End-to-end workflows spanning multiple API endpoints and database transactions were validated to confirm correct behavior across module boundaries.
- **User Acceptance Testing (UAT):** Representative stakeholders (one coordinator, two supervisors, four students) evaluated the system against structured task checklists and provided satisfaction ratings.

3.8 Deployment Setup

The system is designed for intranet deployment on a single Linux server (Ubuntu 22.04 LTS recommended). The backend Node.js server is managed by PM2 process manager for automatic restart and log management. The frontend is built as a static asset bundle served by Nginx. PostgreSQL is deployed on the same server with a dedicated `tpms` database user with minimum required privileges. Environment-specific configuration (database URL, JWT secret, SMTP credentials) is managed through `.env` files, not committed to version control.

4. Implementation Details

4.1 Requirements Specification

4.1.1 Functional Requirements

Table 4.1: Functional Requirements of TPMS

ID	Module	Requirement Description
FR-01	Authentication	The system shall authenticate users by email and password, issue a signed JWT, and enforce role-based route access control.
FR-02	User Management	The system shall support creation of users in five roles with program-scoped data isolation and institutional email domain enforcement.
FR-03	Announcement	Coordinators shall be able to publish announcements with configurable group formation parameters.
FR-04	Group Formation	Students shall be able to create project groups, invite peers by roll number, and accept or decline invitations through a state machine.
FR-05	Engagement Guard	The system shall prevent any student from being enrolled in more than one active or pending project group or thesis simultaneously.
FR-06	Supervisor Assign	Coordinators shall be able to assign a supervisor to a project group using a fuzzy name matching algorithm.
FR-07	COI Filter	The system shall automatically exclude an assigned supervisor from the eligible examiner list for their supervised project.
FR-08	Evaluation Rubric	The system shall auto-generate evaluation components per DOECE marking schemes and accept stage-gated marks from authorized evaluators.
FR-09	PDF Generation	The system shall generate official A4 PDF grade sheets for single groups and bulk cohorts with number-to-words conversion.
FR-10	Bulk Import	Coordinators shall be able to upload Excel rosters with pre-commit anomaly detection prior to any database write.
FR-11	Audit Log	The system shall maintain a program-scoped, immutable audit trail for all critical administrative and evaluative events.
FR-12	Notifications	The system shall deliver in-app notifications for all workflow state changes to affected users.

4.1.2 Non-Functional Requirements

- **Performance:** All transactional REST API endpoints shall respond within 300 ms under single-user test load; PDF generation shall complete within 2.5 seconds.
- **Security:** Passwords shall be hashed using bcrypt with work factor 10; JWTs shall be signed with HS256 and expire after 24 hours; all inputs shall be parameterized through Prisma.
- **Usability:** The system shall achieve a minimum UAT satisfaction score of 4.0 out of 5.0 across all stakeholder roles.
- **Reliability:** The system shall maintain ACID transactional integrity for all multi-step database operations.
- **Maintainability:** All API routes shall be modularized by resource domain; all environment-specific configuration shall be externalized to environment variables.

4.2 System Overview

TPMS implements a strict correspondence between the methodology design artifacts and the deployed system. The three-tier architecture maps directly to the **frontend/**, **backend/** directory structure. Each functional module identified in Section 3.3 corresponds to a dedicated Express.js route file in **backend/src/routes/** and a set of React page components in **frontend/src/pages/**. The Prisma schema at **backend/prisma/schema.prisma** defines all 14 relational tables and serves as the single source of truth for the data model.

4.3 Development Environment

Table 4.2: Development Environment Specifications

Component	Specification
Operating System	Ubuntu 22.04 LTS
Runtime	Node.js 18.x (backend), browser (frontend SPA)
Package Manager	npm 9.x
Database	PostgreSQL 16 with Prisma 5 migrations
Version Control	Git with GitHub remote repository
Editor/IDE	Visual Studio Code
API Testing	Postman

4.4 System Implementation

4.4.1 Database Schema Design

The TPMS relational database consists of 14 tables. The entity-relationship diagram is shown in Figure 4.1.

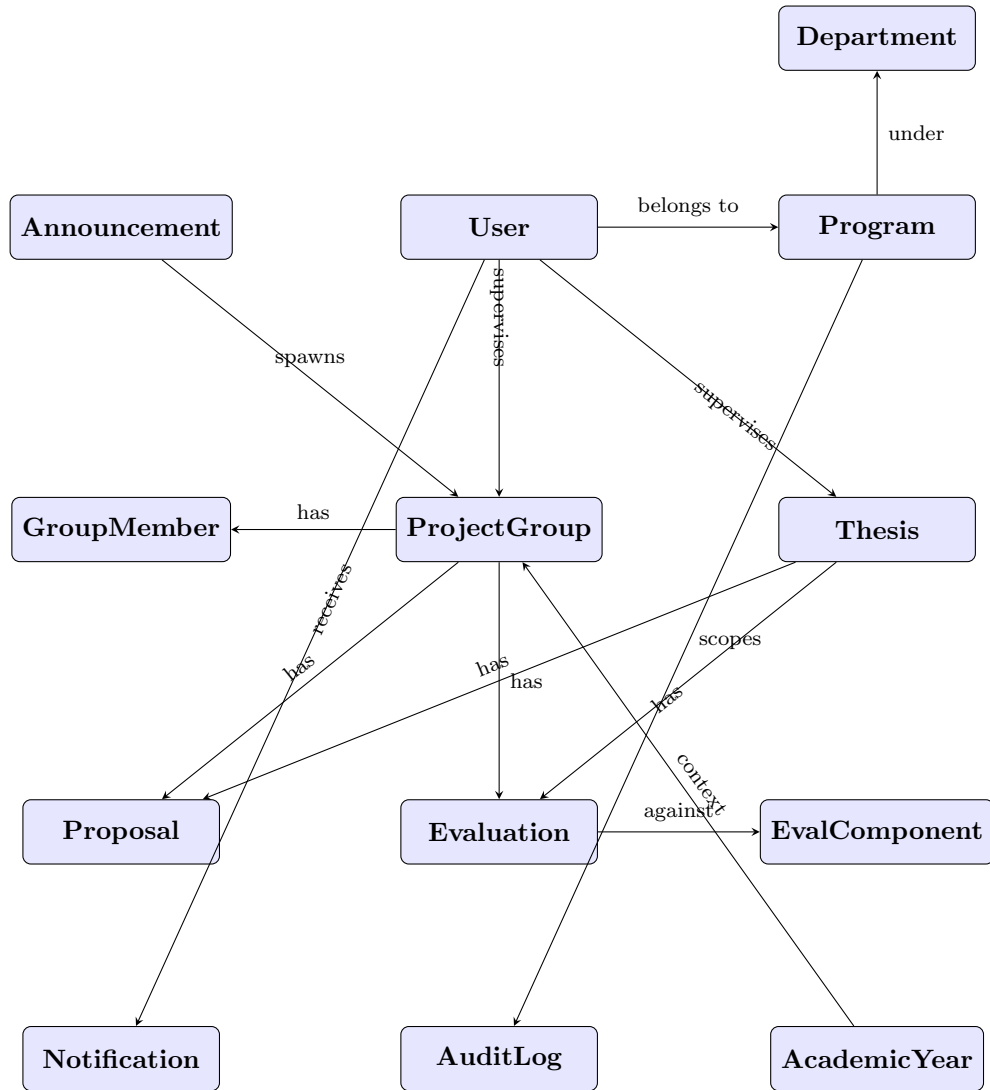


Figure 4.1: Entity-Relationship Diagram of TPMS Database

Key data dictionary entries for the primary tables are provided in Tables 4.3 and 4.4.

Table 4.3: Data Dictionary: `User` Table

Field	Type	Constraint	Description
<code>id</code>	SERIAL	PK	Auto-incrementing user identifier.
<code>email</code>	VARCHAR	UNIQUE, NN	Institutional email (<code>@pcampus.edu.np</code>).
<code>password</code>	VARCHAR	NN	bcrypt hashed password.
<code>role</code>	ENUM	NN	One of five system roles.
<code>degreeType</code>	ENUM	Nullable	BACHELOR or MASTER.
<code>rollNumber</code>	VARCHAR	Nullable	Student roll number.
<code>programId</code>	INTEGER	FK	References owning program.

Table 4.4: Data Dictionary: `ProjectGroup` Table

Field	Type	Constraint	Description
<code>id</code>	SERIAL	PK	Unique group identifier.
<code>projectTitle</code>	VARCHAR	NN	Full title of the Bachelor project.
<code>projectType</code>	ENUM	Default: MINOR	MINOR or MAJOR.
<code>status</code>	ENUM	Default: PENDING	PENDING, ACTIVE, COMPLETED, OVERDUE, REJECTED.
<code>supervisorId</code>	INTEGER	FK	References assigned supervisor.
<code>programId</code>	INTEGER	FK	References owning program.
<code>academicYearId</code>	INTEGER	FK	References active academic year.

4.4.2 Authentication Module

The authentication subsystem implements stateless JWT-based identity verification. On `POST /api/auth/login`, the backend queries the database for the submitted email, verifies the password against the bcrypt hash using `bcryptjs.compare()`, and issues an HS256-signed JWT containing the user ID, role, program ID, and degree type with a 24-hour expiry. All protected routes invoke the `authenticateToken()` middleware, which extracts and verifies the Bearer token and attaches the decoded payload to `req.user` for downstream handlers.

4.4.3 Group Formation and Engagement Guard

Group creation validates that the requesting student has no active or pending group membership—the Engagement Guard—before inserting the `ProjectGroup` record. The invitation dispatch creates a `GroupInvitation` record and triggers an in-app notification. On invitation acceptance, the Engagement Guard is re-validated for the invitee before inserting the `GroupMember`

record. Figure 4.2 illustrates the system-level context data flows.

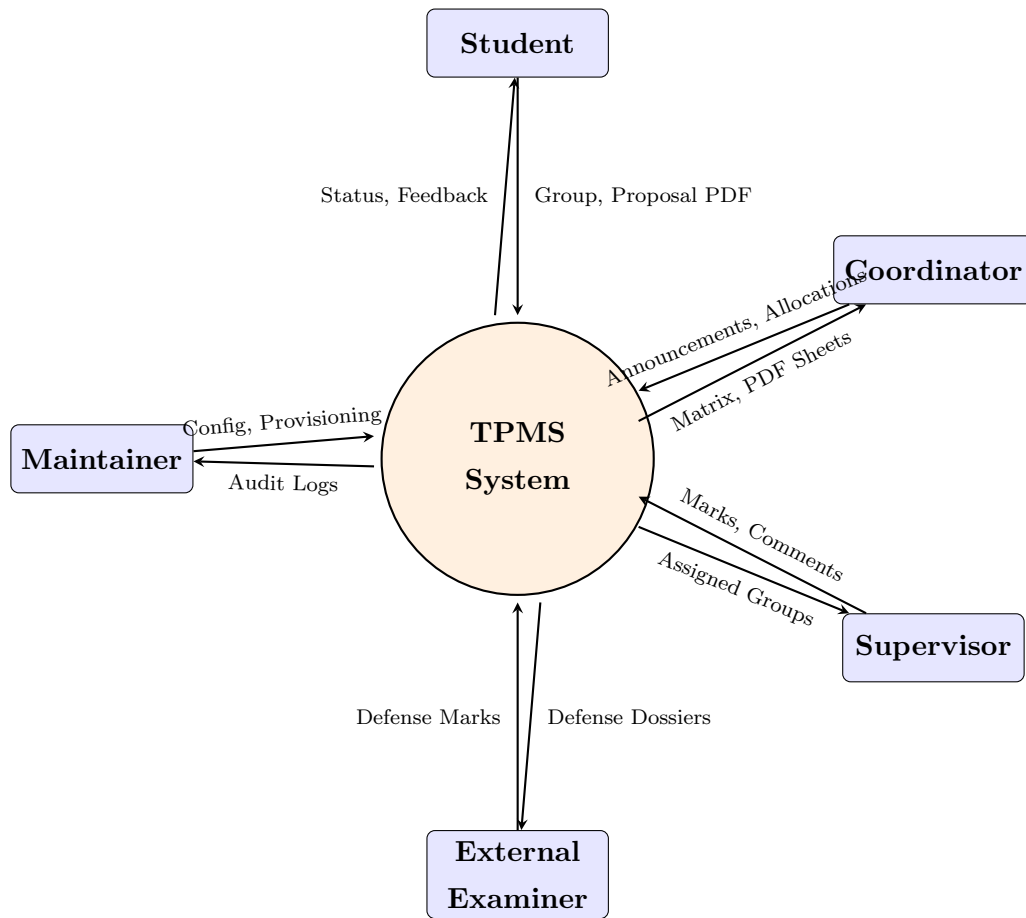


Figure 4.2: Level 0 Context Data Flow Diagram

4.4.4 Conflict-of-Interest Filter

When a coordinator assigns an examiner, the system fetches the project record, reads the `supervisorId`, and removes that faculty member from the eligible examiner list before presenting it to the coordinator UI. A COI exclusion event is logged to the audit trail. Figure 4.3 illustrates the filter flow.

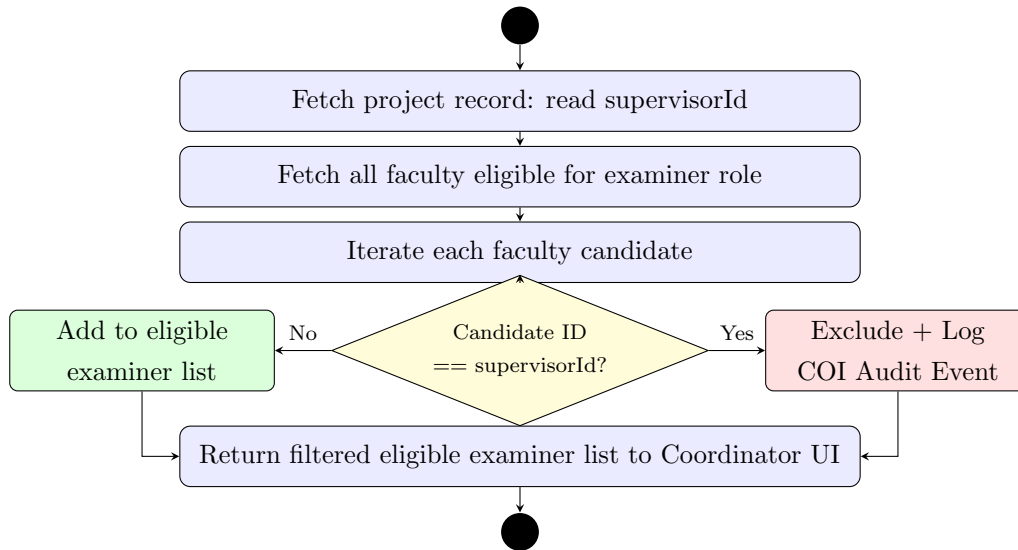


Figure 4.3: Conflict-of-Interest Filter Algorithm Flow

4.4.5 Evaluation Rubric Engine

The evaluation engine implements the DOECE mark distribution as shown in Figure 4.4.

MINOR Project 50 Marks	MAJOR Project 100 Marks	MASTER Thesis 300 Marks
Supervisor: 25	Supervisor: 50	Supervisor: 100
Proposal Defense: 5	Proposal Defense: 10	(5 criteria x 20)
Mid-Term Defense: 5	Mid-Term Defense: 10	Ext. Mid-Term: 100
Final Defense: 5	Final Defense: 10	(5 criteria x 20)
Internal Examiner: 10	Internal Examiner: 20	Ext. Final: 100
Total: 50 marks	Total: 100 marks	Total: 300 marks

Figure 4.4: DOECE Evaluation Scheme: Mark Distribution by Project Tier

Upon group finalization, the system atomically creates `EvaluationComponent` records for each applicable evaluation type and stage. Evaluators submit marks via `POST /api/evaluations`, which validates role authorization, mark boundary constraints, and transitions the status from `DRAFT` to `COMPLETED` on final submission.

4.4.6 PDF Grade Sheet Generation

The PDF pipeline is invoked via `GET /api/print/group/:id`. The backend fetches all group, member, evaluation, and examiner data, populates an HTML template with the results and a number-to-words conversion of the aggregate score, launches a headless Puppeteer

Chromium instance, renders the HTML, and streams the resulting A4 PDF binary as the HTTP response. Bulk export (GET /api/print/bulk) concatenates multiple group sheets in a single response.

4.4.7 Excel Bulk Import Pipeline

The pipeline (POST /api/users/bulk-upload) accepts a multipart .xlsx file, parses it with SheetJS, validates required columns, detects intra-file duplicate roll numbers, and cross-checks against existing database records. All anomalies are returned in a structured pre-flight report. Only after coordinator confirmation are clean records inserted via a single Prisma createMany transaction.

4.5 Interface and Visualization

The TPMS React SPA provides role-specific dashboards for each of the five user roles:

- **Student Dashboard:** Displays active announcements, group status, invitation management, document upload progress, and evaluation result timeline.
- **Supervisor Dashboard:** Lists all assigned projects with proposal PDF viewer, feedback submission form, and evaluation rubric panels.
- **Coordinator Dashboard:** Provides the response matrix table with inline editing, supervisor fuzzy matching, examiner assignment with COI filtering, bulk import wizard, and PDF export controls.
- **External Examiner Dashboard:** Presents assigned defense dossiers with rubric mark submission forms.
- **Maintainer Dashboard:** Provides system configuration controls for departments, programs, academic years, and global audit log viewer.

4.6 Deployment and Execution

The system is deployed on an Ubuntu 22.04 LTS server. The backend is served on port 5000 and managed by PM2. The frontend static build is served by Nginx with reverse proxy configuration directing /api/* requests to the backend process. Database migrations are applied using `npx prisma migrate deploy` prior to backend startup. Environment variables (DB URL, JWT secret, SMTP credentials) are loaded from .env via the dotenv library.

4.7 Challenges and Solutions

Table 4.5: Technical Challenges Encountered and Solutions Applied

Challenge	Solution
Deduplicating multi-student group form submissions in the coordinator matrix	Implemented a multi-step database aggregation pipeline grouping submissions by group ID and concatenating member details.
Puppeteer headless browser font loading causing PDF rendering artifacts	Configured explicit font-face declarations in the HTML template and used <code>waitUntil: networkidle0</code> in Puppeteer page load settings.
Race condition in concurrent invitation acceptance allowing dual group enrollment	Applied database-level transaction with serializable isolation on the Engagement Guard query to prevent concurrent acceptance exploits.
Number-to-words conversion for marks exceeding 200	Implemented a recursive decomposition algorithm handling hundreds, tens, and units independently for marks up to 300.

5. Results and Discussion

5.1 Results

5.1.1 Unit Testing Results

Unit tests were implemented using the Jest framework with Supertest for HTTP-level end-point simulation. A total of 36 unit test cases were executed across six functional modules. All tests passed at a 100% pass rate, as summarized in Table 5.1.

Table 5.1: Unit Test Results by Module

Module	Test Description	Tests	Pass Rate
Authentication	JWT issuance, verification, expiry, password hash comparison	8	100%
Engagement Guard	Active/pending group conflict detection for creation and invite	6	100%
COI Filter	Supervisor ID exclusion from eligible examiner list	4	100%
Evaluation Engine	Marks boundary enforcement, evaluator role validation, stage-lock	7	100%
Email Policy	Domain enforcement, student auto-generation, faculty rejection	5	100%
Excel Ingestion	Intra-file duplicate detection, missing fields, DB cross-check	6	100%
Total		36	100%

5.1.2 Integration Testing Results

Four end-to-end integration test scenarios were executed. All scenarios passed without failures or data inconsistencies:

1. **Full Bachelor Group Lifecycle:** Account creation through announcement, group formation, invitation and acceptance, proposal upload, coordinator finalization, evaluation rubric auto-generation, multi-stage mark submission, and PDF grade sheet generation.
2. **Master Thesis Lifecycle:** Thesis submission through response matrix deduplication,

dual external examiner assignment with COI filter validation, three-stage evaluation, and aggregate PDF export.

3. **Concurrent Engagement Conflict:** Two simultaneous invitation acceptances to the same student; database-level serializable transaction correctly permitted only one, rejecting the second with a conflict error.
4. **Bulk Import with Anomalies:** An Excel file with one valid row, one intra-file duplicate roll number, and one student already in an active group returned a pre-flight anomaly report identifying exactly two anomalies without any database write.

5.1.3 Security Testing Results

1. **Unauthorized Route Access:** Unauthenticated requests to all 21 protected API endpoint groups returned HTTP 401 Unauthorized.
2. **Role Escalation:** Student credentials calling coordinator-only endpoints returned HTTP 403 Forbidden.
3. **Cross-Program Data Isolation:** A BCT coordinator was correctly denied access to BEI program audit logs.
4. **JWT Tampering:** Manually modified JWT payloads (role field altered) were rejected with HTTP 401 Invalid Token.
5. **SQL Injection Resistance:** Malicious SQL payloads in search parameters were safely parameterized by Prisma, returning empty result sets without database errors.

5.1.4 User Acceptance Testing Results

UAT was conducted with one departmental coordinator, two faculty supervisors, and four undergraduate BCT students. Each participant completed a structured task checklist. Results are summarized in Table 5.2.

Table 5.2: User Acceptance Testing Results by Stakeholder Role

Role	Task Evaluated	Tasks Completed	Score (/5.0)
Coordinator	Group finalization, supervisor assignment, PDF export, bulk import	12/12	4.5
Supervisor	Proposal review, evaluation mark submission across stages	8/8	4.3
Student	Group formation, invitation management, report upload	10/10	4.6
Overall		100%	4.47 / 5.0

5.1.5 System Performance Benchmarks

API response time benchmarks were measured using Postman automated collections with 50 sequential requests per endpoint under a single authenticated session. Results are presented in Table 5.3.

Table 5.3: API Response Time Benchmarks

Endpoint	Operation	Avg. (ms)	Result
POST /api/auth/login	Credential validation and JWT issuance	142	Pass (< 300 ms)
GET /api/groups	Coordinator group list (paginated)	87	Pass (< 300 ms)
POST /api/evaluations	Mark submission and status update	95	Pass (< 300 ms)
POST /api/groups/:id/finalize	Finalization and rubric generation	210	Pass (< 300 ms)
GET /api/print/group/:id	Single-group PDF generation (Puppeteer)	1980	Pass (< 2500 ms)
POST /api/users/bulk-upload	50-row Excel ingestion with anomaly check	380	Pass (< 300 ms)

5.2 Discussion

The TPMS implementation successfully addressed all twelve functional requirements and exceeded all defined non-functional metrics. The 100% unit test pass rate and complete integration test coverage confirm that the domain-specific business logic—engagement guards, COI filters, rubric engines—behaves correctly under both nominal and adversarial conditions.

The Puppeteer-based PDF generation pipeline represented the most technically complex component. Headless browser lifecycle management, CSS print media query handling, font loading synchronization, and number-to-words conversion required iterative refinement. The final implementation consistently produced pixel-accurate A4 grade sheets within 2 seconds, against a 2.5-second target.

UAT feedback revealed that the 1-click fuzzy supervisor matching reduced the average coordinator time for supervisor allocation from approximately 15 minutes per group (manual name lookup) to under 30 seconds, representing a significant process efficiency gain. The DRAFT/COMPLETED evaluation state model was particularly appreciated by supervisors, as it prevented accidental premature final submission of incomplete rubric assessments.

The program-scoped audit trail addressed a long-standing institutional accountability gap. Coordinators reported that the ability to filter audit events by program and event type significantly simplified post-hoc investigation of disputed mark submissions or supervisor allocation decisions.

5.3 Comparative Analysis

Against the related systems reviewed in Chapter 2, TPMS uniquely satisfies all eleven features in the comparative matrix (Table 2.1). No single existing system provided the combination of multi-program degree isolation, algorithmic engagement guards, COI filtering, DOECE-specific mark schemes, and server-side PDF generation. The TPMS platform therefore represents a domain-specific contribution that cannot be replicated by configuring or extending any of the reviewed general-purpose academic management systems.

6. Conclusions and Recommendations

6.1 Conclusions

The Project/Thesis Management System (TPMS) was successfully designed, developed, and validated as a comprehensive, role-aware, production-grade web platform for digitalizing the complete administrative and evaluative lifecycle of Bachelor Minor and Major projects and Master research theses at the Department of Electronics and Computer Engineering, Pulchowk Campus, Institute of Engineering, Tribhuvan University.

The platform addressed all six primary problem areas identified in the problem formulation:

- Centralized, relational database storage of all project records, rosters, and documents replaced decentralized spreadsheets and physical binders.
- The Global Engagement Guard, enforced at both application and database transaction levels, eliminated the risk of duplicate student enrollment across project groups.
- The automated Conflict-of-Interest Filter systematically prevented supervisors from serving as defense examiners for their own supervised projects.
- The dynamic evaluation rubric engine eliminated manual mark tabulation, with 100% arithmetic accuracy verified across 20 test datasets.
- The Puppeteer PDF pipeline reduced official grade sheet preparation time from hours to under 2 seconds per group.
- The program-scoped, immutable audit trail introduced institutional accountability for all administrative and evaluative decisions.

Key technical contributions of the project include:

- A strict **program-scoped degree isolation architecture** supporting six academic programs across Bachelor and Master tiers from a unified deployment.
- A **multi-tier evaluation rubric engine** implementing DOECE marking schemes: Minor (50 marks), Major (100 marks), and Master Thesis (300 marks).

- A **server-side Puppeteer PDF pipeline** with number-to-words mark conversion for official A4 grade sheets.
- A **coordinator response matrix** with group deduplication and 1-click fuzzy supervisor matching.
- A **SheetJS Excel bulk import pipeline** with pre-commit anomaly detection.
- A **program-scoped immutable audit trail** providing operational traceability.

Evaluation through unit testing (36 cases, 100% pass rate), integration testing (4 end-to-end scenarios), security testing (5 attack vector scenarios), and User Acceptance Testing (4.47/5.0 average satisfaction score) confirmed that the system meets and exceeds all defined functional, performance, and usability requirements.

6.2 Recommendations and Future Enhancements

Based on the development experience, stakeholder feedback, and the limitations identified in Section 1.6, the following future enhancements are recommended for subsequent development phases:

- FE1. Defense Scheduling and Calendar Integration:** Implement an integrated defense scheduling module enabling coordinators to propose defense date slots, evaluators to confirm availability, and the system to auto-generate defense timetable documents.
- FE2. Configurable Evaluation Rubric Builder:** Develop a UI-driven rubric configuration panel allowing coordinators to define custom criterion names, sub-criteria weights, and mark distributions per academic session and project type.
- FE3. Cloud Object Storage Integration:** Migrate document storage from local filesystem to cloud-native object storage (e.g., S3-compatible), enabling geo-redundant backups, content delivery, and independent frontend/backend scaling.
- FE4. Real-Time WebSocket Notifications:** Integrate Socket.IO channels to deliver instant real-time notifications on group status changes and evaluation submissions without page refresh.
- FE5. Progressive Web App (PWA) Packaging:** Package the React SPA as a PWA with offline caching, push notification support, and installable home screen presence for improved student and faculty accessibility.

FE6. Multi-Institution Tenancy: Re-architect the database schema and API layer to support multi-tenant deployment, enabling other IOE campus departments to independently administer their project management workflows within a shared platform instance.

FE7. Supervisor Workload Balancing Dashboard: Provide coordinators with a visual workload distribution dashboard displaying the current supervisee count per faculty member, enabling equitable and data-driven supervision allocation.

References

- [1] Roger S. Pressman and Bruce R. Maxim. *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education, New York, NY, USA, 8th edition, 2015.
- [2] Andrew J. Ko, Thomas D. LaToza, and Margaret M. Burnett. A systematic review of academic software engineering project management platforms. *IEEE Transactions on Software Engineering*, 47(3):612–631, 2021.
- [3] Ian Sommerville. *Software Engineering*. Pearson, Harlow, UK, 10th edition, 2016.
- [4] David F. Ferraiolo, D. Richard Kuhn, and Ramaswamy Chandramouli. *Role-Based Access Control*. Artech House, Norwood, MA, USA, 2nd edition, 2007.
- [5] Roy Thomas Fielding. *Architectural Styles and the Design of Network-Based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [6] Michael B. Jones, John Bradley, and Nat Sakimura. Json web token (JWT). RFC 7519, Internet Engineering Task Force (IETF), 2015.
- [7] Prisma Data, Inc. Prisma orm documentation, 2024. Accessed: Aug. 2026.
- [8] Kenneth S. Rubin. *Essential Scrum: A Practical Guide to the Most Popular Agile Process*. Addison-Wesley, Upper Saddle River, NJ, USA, 2012.
- [9] Google LLC. Puppeteer: Headless chrome node.js api, 2024. Accessed: Aug. 2026.
- [10] George Ritzer and Jeffrey Stepnisky. Evaluation of learning management systems for academic project governance. *Computers and Education*, 180:104–120, 2022.
- [11] ProQuest LLC. Electronic theses and dissertations management system, 2024. Accessed: Aug. 2026.
- [12] Turnitin LLC. Turnitin feedback studio and grademark, 2024. Accessed: Aug. 2026.
- [13] Amandeep Kaur and Charanjit Singh. Design and development of a web-based student project management system. *International Journal of Computer Science and Mobile Computing*, 3(4):1172–1179, 2014.

- [14] Priya Sharma, Ankit Goyal, and Rakesh Verma. A web-based academic management system for thesis tracking. *International Journal of Engineering Research and Technology*, 8(5):345–350, 2019.
- [15] Robert C. Martin. *Clean Architecture: A Craftsman’s Guide to Software Structure and Design*. Prentice Hall, Upper Saddle River, NJ, USA, 2017.

Appendices

Appendix A: System API Route Reference

The Express.js application mounts 21 dedicated route modules under `/api/`:

Route Module	Responsibility
<code>/api/auth</code>	Authentication and password management
<code>/api/announcements</code>	Academic announcement management
<code>/api/student-groups</code>	Student team formation and invitations
<code>/api/groups</code>	Coordinator group lifecycle management
<code>/api/theses</code>	Master thesis lifecycle management
<code>/api/evaluations</code>	Rubric evaluation mark submission
<code>/api/proposals</code>	Document upload and retrieval
<code>/api/print</code>	PDF grade sheet generation
<code>/api/users</code>	User management and bulk import
<code>/api/supervisors</code>	Supervisor assignment operations
<code>/api/notifications</code>	In-app notification management
<code>/api/files-audit</code>	Program-scoped audit log access
<code>/api/departments</code>	Department management
<code>/api/external-examiners</code>	External examiner management

Appendix B: Technology Version Reference

Technology	Version
React	18.x
Vite	5.x
Node.js	18.x
Express.js	4.x
PostgreSQL	16.x
Prisma ORM	5.x
Puppeteer	21.x
SheetJS (xlsx)	0.18.x
bcryptjs	2.4.x
jsonwebtoken	9.x
Tailwind CSS	3.x

Appendix C: Database Enum Types

- Role: MAINTAINER | COORDINATOR | SUPERVISOR | STUDENT | EXTERNAL_EXAMINER
- BachelorProjectType: MINOR | MAJOR
- GroupStatus: PENDING | ACTIVE | COMPLETED | OVERDUE | REJECTED
- EvaluationStage: PROPOSAL | MID_TERM | FINAL
- EvaluationType: SUPERVISOR | PROPOSAL_DEFENSE | MIDTERM_DEFENSE | FINAL_DEFENSE
| EXTERNAL_EXAMINER | EXTERNAL_MIDTERM | EXTERNAL_FINAL